

Introduction to Deep Learning

Assignment 2

Recurrent neural networks

Myriana Miltiadous(3699463), Pelagia Kalpakidou(3744825), Elena Pitta(3840220)

Introduction

This assignment aims to familiarize us with Recurrent neural networks (RRNs). Specifically, we delved into a sequence-to-sequence problem and tried to develop a neural network that could learn the principles behind simple arithmetic operations (addition, subtraction, and multiplication). Furthermore, the data set we worked on includes textual and image representations from the MNIST data set.

We started by examining query generation code and evaluating the provided initial text-to-text RNN model. Our primary focus was to assess its accuracy and generalization across different training-test set splits, highlighting the network's ability to grasp general principles. We then explored alternative scenarios, such as text-to-image and image-to-text models and to improve performance, we experimented with introducing additional LSTM layers.

Subsequently, we proceed further by attempting to make our networks do multiplication. More specifically, we created models for all two-digit integer multiplications and for these, we utilized our gained knowledge from previous tasks.

Note: All the code along the models discussed in this report, with their exact architectures can be found in the jupyter notebook provided.

1 Part 1: Addition and subtraction

The first part of the project aims to implement three neural networks (text-to-text, image-to-text, and text-to-image) that learn how to add and subtract two integers that are at most two digits long.

1.1 Text-to-Text RNN model

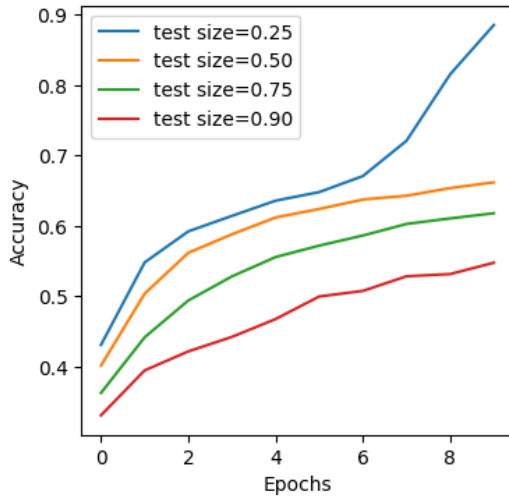
The input of this model is a string of five characters consisting of 2 two-digit long integers and an operand between them. The output is a sequence of 3 characters representing the result of their respective queries.

For this model, we try multiple different splits for the training and test sets. Table 1 shows the test loss and the test accuracy for the four different dataset splits. From the table, we conclude that the splits with a small portion in the train set lead to a lower of accuracy, while the split with 25% test size has the highest accuracy. The model for all the dataset splits was trained for 10 epochs and batch size 10. In addition, Figure 1 shows the results for loss and accuracy in the training. It is clear that the 25% test size has better results in both accuracy and loss.

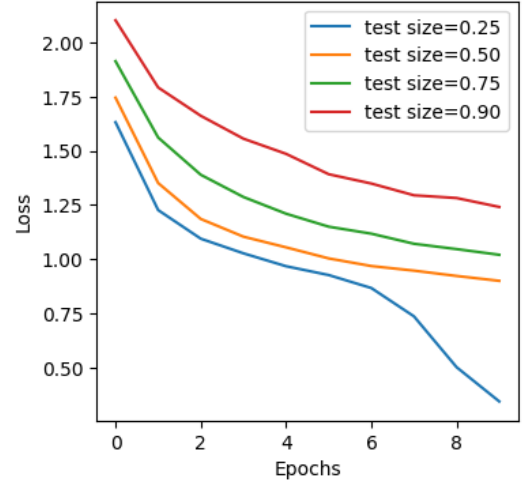
Test split	Test Loss	Test Accuracy
90	1.363	0.499
75	1.015	0.617
50	0.907	0.650
25	0.289	0.916

Table 1: Test loss and test accuracy for different dataset splits.

Table 2 presents 10 wrong predictions for each one of the dataset splits. In the three first columns, the errors seem to be mainly random because during the training there wasn't enough data to train and thus the models can't generalize. Nevertheless, we can draw an interesting conclusion regarding the wrong predictions in the last column where the 25% test size is countered by the highest accuracy rate in the previous table. Noteworthy to



(a) Accuracy during training for different splits.



(b) Loss during training for different splits.

Figure 1: Accuracy and Loss during training for different splits.

highlight that the data are encoded in the following way: $[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, _]$. Also, the output of the model is three vectors of size $(len([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, _]), size = 13)$. Now if we look at the predictions and the true labels of the last column, we can see that the error is at most one position of each of the three vectors. For instance, if we get the query $67 - 7$ the predicted value is 50 and the true value is 60. So, one can easily assume that there is a difference of 10 points from the true value. But in reality is only one wrong position at the vector which represents the decades. Another example is the query $24 - 90$. The predicted value is -67 and the true value is -66 . So the third vector has one position error while the other two vectors are right. This type of error is clearly shown in the last column, however, there are also some examples in the smaller test sizes which are shown with bold in the table.

Table 2: 10 wrong predictions for each multiple test split

Split 0.90			Split 0.75			Split 0.50			Split 0.25		
Opp	Pred	True	Opp	Pred	True	Opp	Pred	True	Opp	Pred	True
73+31	111	104	21-35	-13	-14	49+80	138	129	24-90	-67	-66
11+50	44	61	77-3	71	74	2+9	1	11	1+15	17	16
35-30	1	5	24+60	93	84	46+84	138	130	8+12	19	20
36-85	-47	-49	76-93	-13	-17	87-64	24	33	67-7	50	60
63+73	140	136	83+72	154	155	5+4	1	9	73-79	-7	-6
43-86	-40	-43	28+78	117	106	49-34	11	15	68-61	8	7
42+98	135	140	55+74	133	129	70+15	86	85	97-78	29	19
56+47	111	103	77+92	163	169	34-76	-44	-42	34+5	49	39
39-32	1	7	62-41	22	21	93-19	69	74	11-42	-32	-31
5+23	40	28	78-17	77	61	83+11	90	94	1-92	-90	-91

1.1.1 Adding LSTM layers

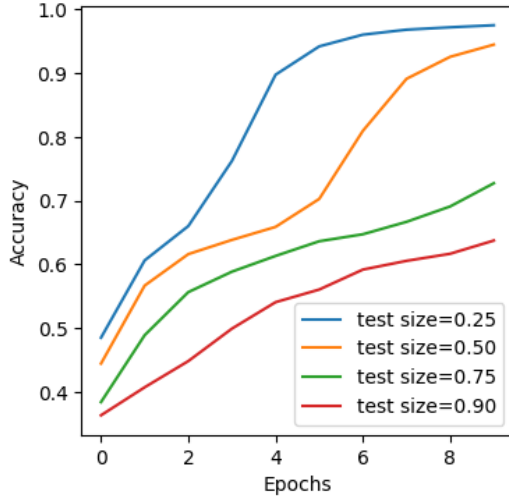
Then a second text-to-text model was implemented with the only difference that it had additional LSTM layers. Specifically, the number of LSTM layers increased by two and also we added two normalization layers. Table 3 presents the results of the test loss and test accuracy for the different dataset splits. In addition, Figure 2 shows the results for loss and accuracy in the training. It is clear that the 25% test size has better results in both accuracy and loss. In comparison with Table 1, it appears that the addition of extra LSTM layers has a positive impact on test loss and test accuracy because decreased and increased respectively.

The reasons for optimizing the loss and test accuracy measures are due to the change in the architecture of the second model. The addition of LSTM layers increases the complexity of the model and allows more complex patterns to be captured during training. Moreover, with the addition of LSTM layers we can extract the conclude

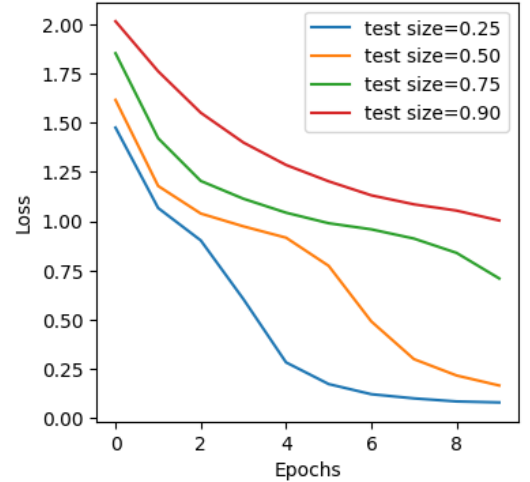
that the second model generalizes better because we have decreased test loss and increased test accuracy in all dataset splits. This means that the first model may not have performed as well on unseen examples because it was not able to learn the underlying patterns in the data as effectively as the second model. Furthermore, the inclusion of the normalization layers in the second model stabilised the training process.

Test split	Test Loss	Test Accuracy
90	1.154	0.575
75	0.715	0.717
50	0.171	0.942
25	0.063	0.981

Table 3: Test loss and test accuracy for different dataset splits for the model with additional LSTM layers.



(a) Accuracy during training for different splits.



(b) Loss during training for different splits.

Figure 2: Accuracy and Loss during training for different splits for the model with additional LSTM layers.

1.2 Image-to-Text RNN model

The input of this model is a stack of images representing a single query with dimensions $[5, 28, 28]$. The output is a sequence of three characters presenting the result of their respective queries. The test size for this model is 25% of the whole dataset (because we got the best results from the previous text-to-text model).

Firstly, the train and test images were flattened as vectors so their shapes are $(15000, 5, 784)$ and $(5000, 5, 784)$ respectively. The first dimension represents the number of samples, the second presents the sequence length and the last dimension represents the flattened version of each image ($28 * 28 = 784$).

Then the model was implemented by using regular LSTM (Long Short-Term Memory) layers for both encoding and decoding. Specifically, the model starts with an encoder that processes a sequence of images and follows with a decoder that generates the strings containing answers of length 3. The architecture of the model is the same as the initial text-to-text model and specifically consists of two LSTM layers from which the first one has as input shape: $input_shape = (None, 784)$, one Repeat Vector layer, and one Time Distributed Dense layer. The model was trained for 25 epochs with Adam optimizer, categorical entropy as loss and batch size of 10.

1.2.1 Adding LSTM layers

Then two more image-to-text models were implemented with their differences concerning the number of LSTM layers. The first of the two models has one extra LSTM layer and the second one has two extra LSTM layers in relation to the initial model.

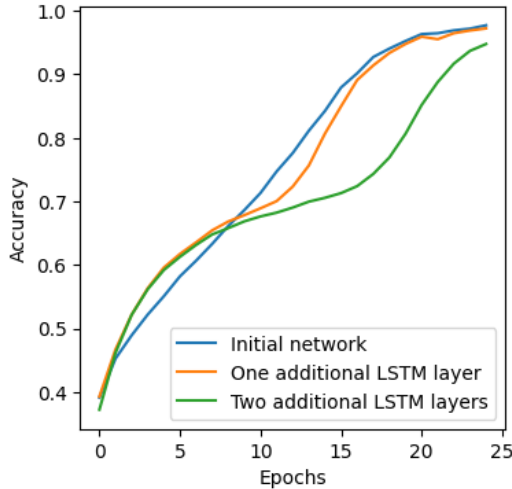
Table 4 presents the test loss and test accuracy metrics for the three image-to-text models, while Figure 3 depicts the accuracy and loss during the training process. An analysis of these results reveals noteworthy information.

Firstly, the initial model presents the lowest test accuracy and the highest test loss compared to the models with additional LSTM layers. This mismatch suggests that the initial model overfits, as it has the best performance on the training data. Furthermore, the addition of additional LSTM layers helps to improve performance in both accuracy and loss measurements and highlights the effectiveness of increased model complexity in capturing complex patterns within the data, while at the same time shows the ability of the models to generalize more effectively to unseen data.

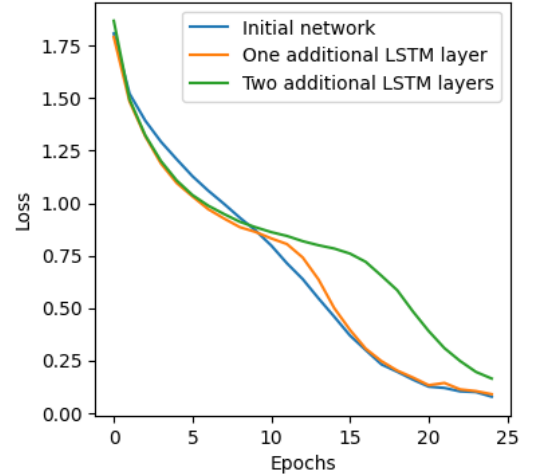
Furthermore, by comparing the 4 models (text-to-text initial, text-to-text with two extra LSTM layers, image-to-text initial and image-to-text with two extra layers) it can be concluded that the text-to-text models perform better. Specifically, the text-to-text model with the addition of two extra LSTM layers stands out as the most effective among the considered architectures.

Model	Test loss	Test accuracy
Initial	3.358	0.505
One extra LSTM	0.926	0.803
Two extra LSTM	0.778	0.822

Table 4: Test loss and test accuracy for three image-to-text models.



(a) Accuracy during training for the three models.



(b) Loss during training for the three models.

Figure 3: Accuracy and Loss during training for the three models.

1.3 Text-to-Image RNN model

Our next model is the text-to-image RNN. It utilizes a sequence-to-sequence architecture, employing Long Short-Term Memory (LSTM) layers for both encoding and decoding. More specifically, the model starts with an encoder that processes input text queries and continues with a decoder that generates a sequence of images. The generated images represent the answer to the arithmetic expression. Furthermore, the text input has length 5 (e.g. '22 + 10') and the output is a set of 3 images with dimensions 28 x 28 ([3, 28, 28]).

The encoder layer consists of a single 512-unit (memory cells within the recurrent layer) LSTM layer followed by Layer Normalization, which captures contextual information from the input text query. As for the decoder layer, it starts with a Repeat Vector layer, which prepares the context vector for image generation. Again, an LSTM layer with 512 units and Layer Normalization is then used to decode the corresponding vector into images. Finally, for image generation, we used a Time Distributed Dense layer, a Reshape layer, and three Conv2DTranspose layers.

To explain that part of the model architecture further, the Time Distributed Dense layer is essential for producing pixel values for each position in the output images. The Reshape layer, which comes after, converts the flat vector output into images with dimensions. In this task, our target dimensions are 3 images 28 x 28 pixels. Lastly, three Conv2DTranspose layers allow for an increase in the spatial dimensions. Taking into account the fact that we aim to generate grayscale images by the final Conv2DTranspose layer, we employed the sigmoid activation that takes

values in the range 0 to 1. For training stability, we applied batch normalization after each Conv2DTranspose layer. These layers work together to convert LSTM outputs into a three-part set of meaningful images representing correct answers.

The model was trained over 20 epochs with the Adam optimizer and a mean squared error loss function. The training set received 75% of the data, while the test set received the remaining 25%. The text and image input data were both preprocessed: the text data were represented in one-hot encoding, and the image data were scaled to values ranging from 0 to 1. On both the training set and a separate testing set, quantitative metrics such as mean squared error (MSE) and mean absolute error (MAE) were recorded to evaluate the learning process. In this case, using accuracy as a metric would not be appropriate for image generation evaluation. To be able to evaluate our model’s performance further, we tested it by generating images for specific textual queries. Here are some examples (Figure 4):

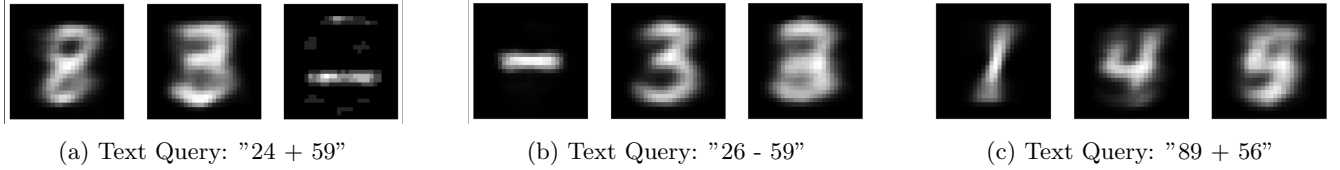


Figure 4: Text-to-image model output with one LSTM encoder layer

The generated images for the text queries we tried are not clear but they represent the correct answer. Converting textual descriptions into clear visual representations while omitting overfitting was challenging. However, the numeric evaluation of the model is an evidence that we created a well trained model. On the training set, the model performed effectively, with an MSE of 0.0402 and an MAE of 0.0919. Similarly, the MSE and MAE on the test set were 0.0419 and 0.0934, respectively. While numerical evaluation indicates a well-trained model, additional experimentation with architecture and hyperparameter tuning may improve image output clarity and lower MSE values.

1.3.1 Adding a LSTM layers

In an attempt to enhance model performance, we experiment with adding additional LSTM layer(s) to the encoder. Subsequently, we are going to examine the performance of four modified models and do a comparison with our first model (one LSTM layer in the encoder).

Modified Model 1: For that model, we added one more LSTM layer of 512 units after the initial one in the encoder part of our basic model. Hence, the architecture of the first modified model consists of two LSTM layers of 512 units for the encoder, followed by layer normalization, and one 512-unit LSTM layer for the decoder. In the encoder part, the first LSTM layer has the setting `return_sequences=True`. The rest of the model’s architecture remains consistent.

Modified Model 2: We followed the same path as before in the second modified model, adding one more LSTM layer, this time with 256 units, after the initial LSTM layer in the encoder. The second modified model’s overall architecture consists of two LSTM layers in the encoder—the initial 512 units and 256 units, followed by layer normalization. The decoder remains unchanged, with a single 512-unit LSTM layer. The first LSTM layer in the encoder has the `return_sequences=True` setting, just like the first modified model.

Modified Model 3: We adopted a different approach with the third modified model. Instead of increasing or decreasing the number of units in the added LSTM layer, we added two additional LSTM layers after the initial encoder LSTM layer, both with 512 units. The third modified model’s overall architecture consists of three LSTM layers in the encoder—512 units, 512 units, and the initial one—followed by layer normalization. In this case, the encoder’s first and second LSTM layers keep `return_sequences=True`.

Modified Model 4: For this model, we extended the investigation by adding again two additional LSTM layers but with different unit settings. The first LSTM layer with 256 units is added, followed by a second layer with 128 units. Hence, the fourth modified model’s overall architecture consists of three LSTM layers in the encoder—256 units, 128 units, and the initial one—followed by layer normalization. Once again, the decoder uses a single LSTM layer of 512 units. The first and second LSTM layers in the encoder, like the previous model, have the `return_sequences=True` setting.

In Table 5 we have the training and test set MSE and MAE at 20 epochs for the initial and four modified models. In the text-to-image model evaluation, we decided to represent the outcomes in a table because the plots

picturing MSE and MAE on 20 epochs were not informative. In that small number of MSE and MAE during epochs, the changes were minor and not clear when plotting them.

Model	Training MSE	Test MSE	Training MAE	Test MAE
Initial	0.0402	0.0419	0.0919	0.0934
One extra LSTM 512 units	0.0391	0.0418	0.0895	0.0934
One extra LSTM 256 units	0.0390	0.0414	0.0896	0.0915
Two extra LSTM both 512 units	0.0412	0.0415	0.0941	0.0940
Two extra LSTM 256 & 128 units	0.0413	0.0420	0.0945	0.0948

Table 5: Training and Test MSE and MAE for five text-to-image models.

The goal behind these modifications was to see how increasing the depth and complexity of the LSTM layers in the encoder affects the model’s performance. More specifically, the training and test MSE and MAE of these four modified models were evaluated and compared to the initial model with a single LSTM layer in the encoder. The variations in model performance as given in Table 5, can be explained by the trade-off between model complexity and generalization. For the modified model with one additional layer, the increase in complexity improved performance. Especially for the Modified Model 2, with the one additional 256 units LSTM layer, we observe a significant performance improvement (Test MSE: 0.0390, Test MAE: 0.0896). However, increasing complexity did not result in corresponding gains for Modified Models 3 and 4 with the two additional LSTM layers. Overall, the weaknesses of the initial model are captured when comparing it with the MSE and MAE values reached by more complex architectures. Our initial model could not succeed a low values for both MSE and MAE metrics on both training and test sets indicating difficulties in the training process. Although the improvement that the addition of one layer showed, we cannot overlook the drawback of the excessive complexity introduced by the addition of two additional LSTM layers. The risk of overfitting, as shown by the MSE and MAE values across training, test, and validation sets, highlights the difficulties in training the network effectively as complexity increases.

1.3.2 More epoch for image appearance improvement

In an attempt to improve image quality, we increased the number of training epochs, which revealed overfitting issues in our top-performing models. However, after 50 epochs, we saw significant improvements in image clarity, as shown in Figure 5.

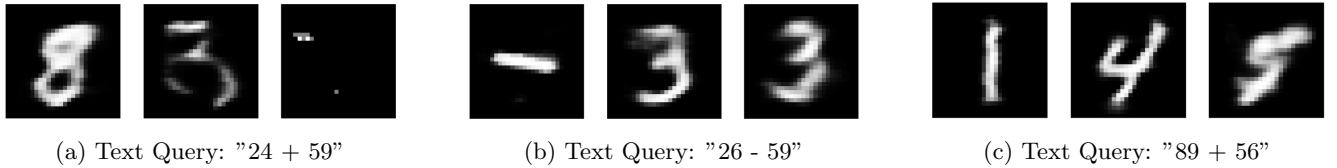


Figure 5: Text-to-image model output with best performed model - 50 epoch

Based on the results in Table 5, we chose three models to investigate further: the initial model with a single LSTM layer, Modified Model 2 with an additional 256-unit LSTM layer, and Modified Model 3 with two additional 512-unit LSTM layers. These models performed highly within their respective architectures.

Model	Training MSE	Test MSE	Training MAE	Test MAE
Initial	0.0274	0.0488	0.0648	0.0902
One extra LSTM 256 units	0.0172	0.0555	0.0443	0.0897
Two extra LSTM both 512 units	0.0208	0.0528	0.0516	0.0897

Table 6: Training and Test MSE and MAE for five text-to-image models on 50 epochs

The subsequent examination of MSE and MAE values for 50 epochs clarifies further the models’ behavior (Table 6). Specifically, the initial model improved significantly in training MSE (from 0.0274 to 0.0172), indicating effective learning and weight adjustment. However, the increase in test MSE (0.0488 to 0.0555) indicates potential overfitting, which limits generalization. Furthermore, while training MAE decreased (0.0648 to 0.0443) which

indicates that the accuracy increased, the test MAE remained relatively stable (0.0897). Moving forward to the model with an additional 256-unit LSTM layer showed promise with a lower training MSE (0.0172) but a higher test MSE (0.0555), indicating potential overfitting. Additionally, training MAE improved significantly (0.0443), whereas test MAE remained very similar (0.0897). The model with two additional LSTM layers at 512 units produced more accurate results, with higher training MSE (0.0208) but lower test MSE (0.0528), indicating improved generalization. Despite the increase in training MAE (0.0516), indicating difficulties in predicting errors during training, the test MAE (0.0897) remained stable. The 50-epoch results revealed overfitting challenges for our model. A potential solution would have been to add drop-out layers or early stopping to optimize the trade-off between training accuracy and generalization in these models. Overall, taking into account the complexity of the text-to-image model, Modified Model 3 with two additional 512-unit LSTM layers appears to be the most balanced option, demonstrating improved generalization while having a lower test MSE.

2 Part 2: Multiplication

After the thorough examination of the numerous models created to solve addition and subtraction problems, the focus of this report now shifts towards developing models for a more complex mathematical operation—multiplication. Similar to the methodology employed in Part 1, the dataset for multiplication is constructed utilizing the MNIST dataset along with a specialized function generating visual representations of the `*` symbol.

In this process, five images are stacked to generate a combined representations of multiplications between two numbers ranging from 0 to 99. Thus, the resultant images take the shape of (5, 28, 28), and each of them is associated with its corresponding text representation and the result of the multiplication operation. For instance, if a picture visually represents the mathematical operation 28×79 , it is linked to its textual equivalent `'28*79'` and the resultant value of operation `'2212'`. A padding method is used to account for cases where one or both of the numbers involved in the operation are not two-digit numbers (e.g. `'0*8 '`, `'9*81 '` etc).

Following this methodical technique, the multiplication data set is built, comprising a collection set of 10,000 samples representing up to two-digit integer multiplications.

Utilizing the stacked images along with their respective mathematical operations and resultant outcomes, the objective is to apply Recurrent Neural Networks (RNNs) for both text-to-text and image-to-text tasks, predicting the associated results.

2.1 Text-to-Text RNN models

Starting with the text-to-text Recurrent Neural Network (RNN) models, the goal is to give as an input a text string such as `'2*8'` and train the network to predict the associated output, which is `'16'` in this case. To do so an encoding function was utilized, which converts text labels into one-hot encoded vectors. For each character in a label, it maps the character to a unique index using a dictionary. Then, it creates a binary one-hot vector representation for each character. The resultant one-hot encoded vectors are concatenated to create a three-dimensional array. The first dimension corresponds to the number of samples (10000), while the second and third dimensions are 5 and 12, respectively. The number 5 denotes the five different elements of the sequence, and 12 represents the total count of distinct symbols, including padding (`'0123456789* '`), in the sequence of interest.

So, this text-to-text model processes the one-hot-encoded strings of five characters, similarly with the structure of addition/subtraction problems, previously discussed. The model, then, produces an output one-hot-encoded sequence with shape (5,12). The choice of 5 stems from the maximum result of a multiplication between two two-digit numbers, which is `'-9801'`, resulting from the multiplication between -99 and 99. The choice of 12 again denotes the twelve unique characters of this problem. After decoding this output, the result of the mathematical operation should be given. To assess performance, several simple models, each featuring an identical architecture as described in part 1, were trained for 20 epochs across three distinct train-test splits. The decision to extend the training duration to 20 epochs, in contrast to the 10 epochs employed for the addition/subtraction problem, was motivated by the comparatively greater complexity of the multiplication task. The referring model architecture, again consists of a sequential model initialized to encode input sequences with an LSTM layer of 256 units, followed by a RepeatVector, another LSTM and finally a TimeDistributed layer with a softmax activation that predicts the output characters at each time step. The model is again compiled with categorical crossentropy loss and the Adam optimizer. Finally, batch size was chosen to be 10.

Figure 6a depicts training and validation accuracy and 6b training and validation loss. The figures indicate that overfitting becomes noticeable after around 10 epochs. As expected, models with smaller test sizes demonstrate superior performance on the training set. Notably, the model with a 25% test split stands out as the top performer

on the test set. This is due to the fact that, while the model with the smallest test split of 10% displays impressive performance on the training set, it suffers to overfitting. As a result, its validation and test metrics are slightly worse, compared to the model with a test split of 25%.

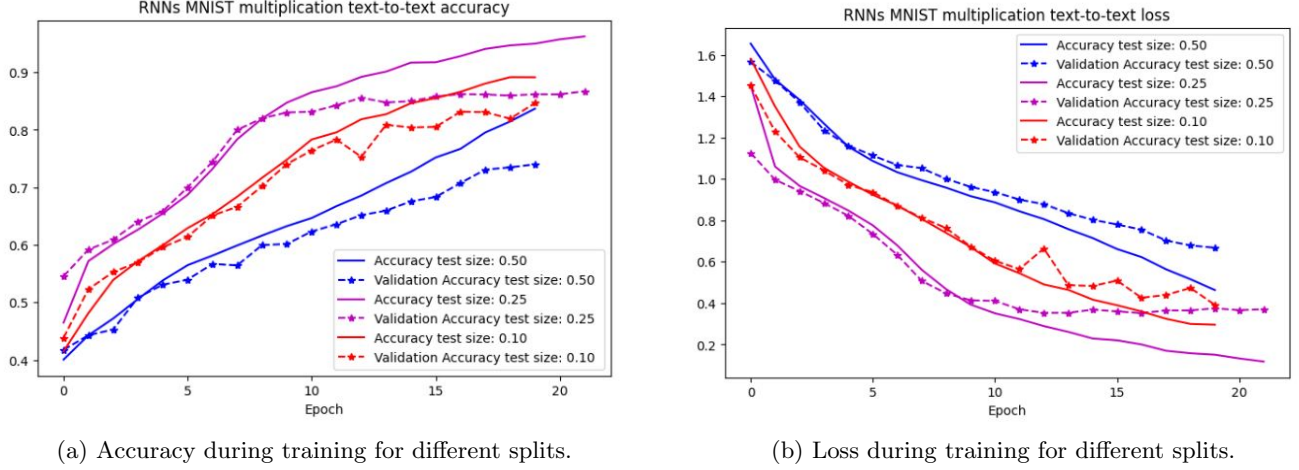


Figure 6: Multiplication problem: Accuracy and Loss during training for different splits.

In Table 7, the test loss and test accuracy are additionally presented after the 20 epochs of training. However, the corresponding test metrics for 10 epochs are provided for the model with 0.25 test split, which is the best performed model of both this part and part 1. This allows for a direct comparison between the outcomes of identical models, trained on the two different problems of multiplication and addition/subtraction. Since, as already highlighted, multiplication is a more complex operation, it was expected that the model trained for the addition/subtraction problem would perform better, which it did, since it has biggest accuracy of 0.916 and lower loss of 0.289 on the test set, compared to the model trained for multiplication, which has a test loss of 0.731 and a test accuracy of 0.715. Hence, the model of part 1, has better generalization capabilities.

Test split	Test Loss	Test Accuracy
10	0.399	0.839
50	0.660	0.746
25	0.390 (0.731)	0.843 (0.715)

Table 7: Multiplication problem: Test loss and test accuracy for different dataset splits.

Tune architecture and hyperparameters

The next step we followed, was to fine tune hyperparameters and architectures of different text-to-text models, in order to find the one that yields the best performance on test set. The key idea of the experiments conducted, was to add one layer each time and see how the respective model performance on the test set changes. All the models were trained with a 90% split on training data and 10% validation split, in order to experiment with reducing overfitting (observed for this split) and achieve a better performance on test set.

The first model tested, has an increased number of LSTM units in the initial layer, aiming to capture more intricate patterns in the input sequences. Then a normalization layer was added, to contribute to the model’s stability during training, potentially aiding in faster convergence and better generalization. Additionally, an extra LSTM layer is inserted, accompanied by layer normalization and dropout with a rate of 0.2. The model retains, the previously mentioned, RepeatVector, LSTM and TimeDistributed layers. This model, managed after 10 epochs of training to achieve a test loss of 0.348 and a test accuracy of 0.862.

The next model we trained, again for 10 epochs, has an additional LSTM layer with 512 units after the dropout of the previously mentioned model. That model performed poorer with test loss of 0.669 and test accuracy of 0.710. A possible reason for that might be the increased danger of gradient vanishing or exploding, which is more likely in deeper architectures with many LSTM layers.

Since the model with two additional layers did not perform well, we stop adding any more layers and start experimenting with the number of units per LSTM layer in the first model mentioned in this subsection. After

experimenting with a variety of combinations of the numbers 64, 128, 256 and 512, the final model that was identified as the one with the best performance was trained for 30 epochs and achieved to have 0.289 loss and 0.898 accuracy on test data. Its training and validation performance are shown in figure 7. The first two LSTM layers of the model, to achieve such performance, should have 256 units and the third one 512. Finally a drop out rate of 0.25 should be applied.

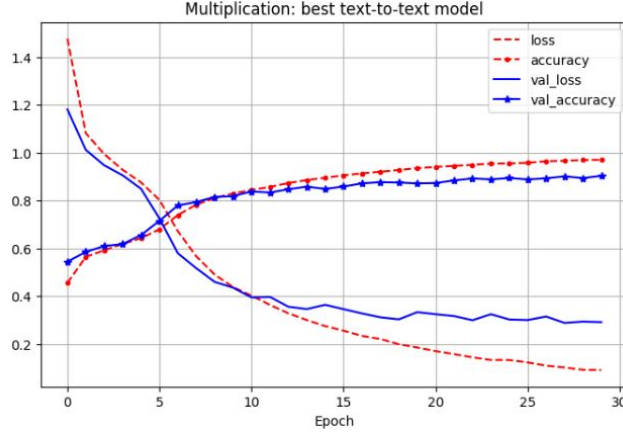


Figure 7: Best model for multiplication problem

2.2 Image-to-Text RNN model

Continuing with the multiplication problem, we conducted training on image-to-text models and evaluated their performance on a test set. These models receive input elements of dimensions (5,784), where the first dimension, 5, corresponds to the maximum possible result of two-digit integer multiplication (as previously explained). The second dimension, 784, represents the flattened image with a shape of (28,28), illustrating either a numeric digit or the symbol '*'. The output format is identical to that of the text-to-text models. All models were trained with a test set split at 25% and a validation set split at 10%. To reduce overfitting, an early stopping criterion was used, with a patience of 5 epochs and loss monitoring during training.

As with our prior methodology, we begin with the simplest model, similar with the one used in the text-to-text context. The distinction lies in the model's adaptation to process input data formatted as (5, 784) this time. After undergoing training for 10 epochs, early stopping was conducted, resulting in a test loss of 1.337 and a corresponding test performance of 0.486. This is a rather poor performance so we proceed by adding extra layers.

The same procedure as described in the text-to-text scenario was followed by exploring exactly the same models with one and two additional LSTM layers, layer normalization, dropout and the exploration of variable units per layer. On the test data, the model with the best performance had a loss of 0.951 and an accuracy of 0.706. Before triggering an early stop, this model was trained for 42 epochs. This model had two additional LSTM layers and the units per LSTM starting from the input layer were 128, 128, 256, 256.

The majority of the other models tested had an accuracy around 0.55 and a loss around 1.2 so the performance of the best aforementioned model was considered sufficient.

Finally, again compared to the image-to-text models for addition/subtraction, the multiplication models perform poorer suggesting their more limited ability of generalization. This applies, even if we compare the two best models of the two problems.

2.2.1 Discussion

While the accuracy and loss achieved by the top-performing multiplication models were considered sufficient, the inherent complexity of multiplication motivated us to push the boundaries and aim for even more improved accuracy. In pursuit of this goal, we devised image-to-text models specifically designed to predict the text depicted in an image rather than focusing on the outcome of the mathematical operation within the photo.

To accomplish this, we employed a synergistic approach by combining Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN), constructing various models to assess their individual performances. For training those models some additional preprocessing of the data was performed. The first step was to horizontally stack the five images representing a multiplication operation, resulting in composite images with dimensions of (28,140). After that, the image pixel values were normalized and the textual labels were encoded, with the same encoding as explained previously. So the models tested take as input an image and give as an output the text that appears in that image, but in the encoded format, which after being decoded would be something like '78*25'.

We experimented with models with varying numbers of layers and units per layer. The training and testing sets were split 80:20, and the models were trained for 32 epochs. The more effective model produced notable results, with a test loss of 0.054 and a test accuracy of 0.985. These outcomes were seen to be noteworthy.

The architecture that was chosen as the best performed one, consists of three convolutional layers with batch normalization, dropout, and max-pooling. A reshape layer is employed to prepare the data for Long Short-Term Memory (LSTM) processing, assuming five time steps. The LSTM layer is configured with 128 units and returns sequences. Two time-distributed dense layers follow, incorporating ReLU and softmax activations, respectively. The model is compiled using the Adam optimizer and categorical crossentropy loss. Training for 10 epochs with a batch size of 32, followed by evaluation, produced good results, as evidenced by the reported test loss and accuracy.

Given that the reported accuracy of 0.985 is character-level accuracy, focusing on correctly identifying individual numbers in each picture, we opted to extend the evaluation to also account for correctly predicting multiplication results. We computed the result of each text sequence output and compared it to the actual result of multiplication by leveraging the model's decoded predictions for the test set and a function that can calculate results of mathematical operations given as text. Once again, the results were outstanding, with a 92.85% accuracy in properly anticipating multiplication outcomes.

Conclusion

In conclusion, the investigation of encoder-decoder recurrent models for arithmetic operations such as addition, subtraction, and multiplication revealed the models' adaptability and learning abilities. The text-to-text LSTM model demonstrated its ability to comprehend underlying arithmetic principles without relying on memorization. Variations in train-test splits demonstrated the model's generalizability, while a focused evaluation revealed specific misclassification patterns. The use of image-to-text and text-to-image models broadened the scope, successfully predicting outcomes from MNIST images and generating image sequences as text-query answers. The models demonstrated their adaptability as the complexity of the multiplication task increased, prompting a deeper dive into architecture tuning to optimize accuracy. This in-depth investigation provides valuable insights into the nuanced dynamics of recurrent models in solving a variety of arithmetic challenges, paving the way for future advances in mathematical learning and generalization.